GPUMODE Lecture Study Notes: Lecture 102: quartet v2

Core Problem the Lecture Addresses

The lecture studies how to train large language models using extremely low precision arithmetic, especially **NVFP4** on modern NVIDIA GPUs, while preserving the optimization behavior of higher precision training. The central problem is not merely how to store values in four bits, but how to make the entire training computation stable, accurate, and fast enough that the theoretical hardware speedup becomes useful in practice.

The key bottleneck is the following tension:

> Four-bit tensor cores can provide much higher matrix multiplication throughput than BF16 or FP16, but naive FP4 quantization introduces large biased errors. Training only works if the quantization procedure preserves the useful direction of the gradient while keeping quantization error low enough to avoid quality loss.

The talk focuses on **quartet v2**, a recipe for end-to-end NVFP4 training of linear layers in large language models. The method improves over earlier NVFP4 training recipes by replacing expensive per-element stochastic rounding in FP4 with a more structured, lower-error method based on randomized Hadamard rotations, Eden-style unbiased quantization, and stochastic rounding only on FP8 micro-scales.

At a high level, the method tries to solve four connected problems:

1. **Unbiased gradient estimation:** low precision backward-pass quantization should not systematically push gradients away from the true higher precision gradient.
2. **Low quantization error:** unbiasedness alone is insufficient if the estimator has very high variance or mean squared error.
3. **NVFP4 compatibility:** the scheme must fit NVIDIA's three-level FP4 format, with FP4 values, FP8 micro-scales, and a global scale.
4. **Efficient GPU implementation:** the method must fuse dequantiza-

tion, randomized rotation, transpose, scale estimation, quantization, and correction into custom kernels to avoid excessive global memory traffic.

Required Background

Why Low Precision Training Matters

Modern accelerators expose multiple floating point formats, ranging from FP64 down to FP4. The reason to use lower precision is simple: lower bitwidths reduce memory movement and allow tensor cores to execute more arithmetic per cycle.

If an operation is dominated by matrix multiplication, and the hardware supports a lower precision tensor core path, then idealized throughput often scales approximately inversely with bitwidth:

$$\text{Throughput gain} \approx \frac{\text{old bitwidth}}{\text{new bitwidth}}.$$

For example, moving from 16-bit arithmetic to 4-bit arithmetic has a potential arithmetic throughput gain of roughly

$$\frac{16}{4} = 4.$$

This does not mean end-to-end training becomes $4\times$ faster, because many other costs remain: quantization kernels, optimizer updates, normalization, communication, kernel launch overhead, and framework overhead. But it explains why FP4 training is worth pursuing.

Historical Progression of Training Precision

The lecture places NVFP4 training in a historical sequence.

FP32 Training

In traditional FP32 training, the model weights, gradients, optimizer states, and most computations use full precision. This usually works without special numerical tricks:

$$\theta_{t+1} = \theta_t - \eta \cdot \widehat{g}_t,$$

where θ_t are parameters, η is the learning rate, and $\widehat{g}_t$ is the gradient estimate.

FP16 Training

FP16 reduced memory traffic and increased tensor core throughput, but introduced range issues. FP16 has a narrower dynamic range than FP32, so small gradients can flush to zero. A standard trick is **loss scaling**. Instead of backpropagating the loss L, one backpropagates

$$L' = sL,$$

where s is a scale factor. The resulting gradients are divided by s before the optimizer step:

$$g = \frac{1}{s}\nabla_\theta L'.$$

A second common trick is to keep **master weights** and optimizer statistics in FP32, while using FP16 mainly for matrix multiplications.

FP8 Training

FP8 training requires more than a simple cast. Because FP8 has limited precision and range, values are commonly quantized in groups with explicit scale factors. A groupwise quantization scheme can be written as

$$q_i = Q_{\mathrm{FP8}}\left(\frac{x_i}{s_g}\right), \qquad \widehat{x}_i = s_g q_i,$$

where s_g is a scale shared by group g. The scale artificially extends the representable range.

DeepSeek-style FP8 training and related recipes combine low precision tensor core operations with higher precision accumulation. The multiplication may use FP8 operands, but partial sums are accumulated in higher precision.

FP4 Training

FP4 is much more difficult. The representable values are extremely sparse. A simplified E2M1-like FP4 set contains values similar to

$$\{0,\ \pm 0.5,\ \pm 1,\ \pm 1.5,\ \pm 2,\ \pm 3,\ \pm 4,\ \pm 6\}.$$

The gap between adjacent representable values can be large. For example, representing 5 requires choosing between 4 and 6, producing a large absolute error. Consequently, a successful FP4 training method needs:

- groupwise scales;

- outlier handling;
- randomized transforms;
- unbiased gradient estimation;
- carefully fused kernels;
- higher precision optimizer states.

Microscaling Formats

Microscaling formats use a low precision value per element and a shared scale per small group of elements. A generic microscaling representation is

$$x_i \approx s_g q_i,$$

where q_i is a low precision value and s_g is the scale for group g.

The Open Compute Project standardized microscaling formats such as MXFP4. NVIDIA's NVFP4 is related but more specialized. It uses a three-level structure:

$$x_i \approx S_{\text{global}} \cdot s_g \cdot q_i,$$

where

- q_i is an FP4 value;
- s_g is an FP8 micro-scale, often shared by 16 elements;
- S_{global} is a higher precision global scale.

This gives NVFP4 better range control than a pure two-level format.

Main Concepts

NVFP4 Representation

The NVFP4 representation can be conceptualized as

$$\widehat{x}_i = S \cdot s_{\lfloor i/16 \rfloor} \cdot q_i,$$

where S is the global scale, $s_{\lfloor i/16 \rfloor}$ is an FP8 scale for a group of 16 values, and q_i is the stored FP4 value.

A typical scaling construction chooses scales so that the maximum magnitude maps to the largest representable FP4 value, which is approximately 6. If m_g is the local absolute maximum in a group, then an idealized local scale is

$$s_g \approx \frac{m_g}{6}.$$

The actual NVFP4 scheme is more subtle because the micro-scale itself must be representable in FP8, and a global scale is also used.

Why Unbiasedness Matters

A quantizer Q with randomness ω is unbiased if

$$\mathbb{E}_\omega \left[Q(x, \omega) \right] = x.$$

For training, the relevant object is often not only a tensor x, but the gradient estimate. If backward-pass quantization introduces systematic bias, then the optimizer may converge to a worse point or become unstable. In stochastic optimization, one usually wants

$$\mathbb{E} \left[\widehat{g}_t \mid \theta_t \right] = \nabla_\theta L(\theta_t),$$

where $\widehat{g}_t$ is the gradient estimator used by the optimizer. FP4 backward quantization can break this property unless the quantization method is designed carefully.

Per-Element Stochastic Rounding

A standard way to obtain unbiased quantization is stochastic rounding. Suppose x lies between two representable grid points a and b, where $a \le x \le b$. Stochastic rounding chooses a or b with probabilities

$$P(Q(x) = a) = \frac{b - x}{b - a}, \qquad P(Q(x) = b) = \frac{x - a}{b - a}.$$

Then

$$\mathbb{E}[Q(x)] = a \frac{b - x}{b - a} + b \frac{x - a}{b - a} = x.$$

This is mathematically attractive, but in FP4 it is expensive in error. Stochastic rounding from 5 to 4 or 6 is unbiased, but the instantaneous error is large. The lecture emphasized that stochastic rounding on every FP4 element can more than double the expected quantization error relative to round-to-nearest.

Round-to-Nearest

Round-to-nearest minimizes local squared error for a fixed quantization grid. If Q_{RTN} denotes round-to-nearest, then for a scalar x,

$$Q_{\mathrm{RTN}}(x) = \arg\min_{q \in \mathcal{G}} |x - q|^2,$$

where $\mathcal{G}$ is the quantization grid. It has lower mean squared error than stochastic rounding, but it is generally biased:

$$\mathbb{E}\left[Q_{\mathrm{RTN}}(x)\right] \neq x.$$

quartet v2 tries to keep most of the low-error behavior of round-to-nearest while recovering unbiasedness through randomized rotations and scale corrections.

Randomized Rotations as a Source of Unbiasedness

The key mathematical idea is to quantize not x directly, but a randomly rotated version of x. Let R_ω be a random orthogonal matrix:

$$R_\omega^\top R_\omega = I.$$

Define the rotated quantization estimator

$$\widehat{x} = R_\omega^\top Q(R_\omega x).$$

If Q were the identity, then

$$R_\omega^\top R_\omega x = x.$$

If Q is a quantizer, error is introduced in rotated coordinates. Geometrically, this can be interpreted as randomly rotating the quantization grid around the vector. For sufficiently symmetric random rotations, every quantization error direction has a matching opposite direction with equal probability. As a result, the expected quantized vector becomes colinear with the original vector:

$$\mathbb{E}_\omega\left[R_\omega^\top Q(R_\omega x)\right] = \alpha x,$$

for some scalar α. This is not yet unbiased unless $\alpha = 1$, but it means the expected direction is correct.

Eden-Style Reprojection

Eden-style quantization adds a correction factor that rescales the quantized vector so that the expectation becomes exactly, or approximately, unbiased. Let

$$z = R_\omega x, \qquad q = Q(z).$$

The correction factor is chosen so that the corrected vector lies on the tangent hyperplane associated with the original vector. A practical scalar correction can be written using an inner product ratio:

$$c = \frac{\langle z, z \rangle}{\langle q, z \rangle}.$$

Then the corrected quantized vector in rotated coordinates is

$$\widetilde{q} = cq,$$

and the de-rotated estimator is

$$\widehat{x} = R_\omega^\top \widetilde{q} = R_\omega^\top c Q(R_\omega x).$$

The intuition is that if q is too short or too long in the direction of z, the scalar c corrects the projection. With symmetric rotations, the perpendicular error components cancel in expectation.

Visual Concept

Draw a two-dimensional vector x, then draw a square quantization grid. Show the vector being rotated into the grid, rounded to a nearby grid point, and rotated back. Then draw the opposite grid rotation that produces an error vector mirrored across x. The average of the two errors lies along the original vector.

Why Direct Eden Reprojection Does Not Fit NVFP4

The Eden correction factor c is usually close to 1, often requiring fine precision. In NVFP4, there are three possible places to apply it:

1. multiply FP4 values q_i;
2. multiply FP8 micro-scales s_g;
3. multiply the global scale S.

Applying it to FP4 values is too coarse. Applying it only to the global scale is too coarse spatially, because the correction is needed per small rotation or quantization group. quartet v2 applies the correction to the FP8 micro-scales with stochastic rounding.

This is the central trick:

> Do not stochastically round every FP4 element. Instead, use mostly round-to-nearest for the FP4 values, compute Eden correction factors, and stochastically round the corrected FP8 micro-scales.

The error reduction comes from two facts:

- FP8 scales are much more precise than FP4 values.
- There is only one scale per 16 FP4 values, so stochastic rounding is applied much less frequently.

quartet v2 Backward Quantization

For a vector x, quartet v2 uses groups of size 128 for randomized rotation and subgroups of size 16 for NVFP4 micro-scales. A simplified form of the method is:

1. Partition the tensor into groups of 128 values.
2. Apply a randomized Hadamard rotation H_ω within each group.
3. Quantize the rotated values to FP4 mostly using round-to-nearest.
4. For every group of 16 values, compute an Eden correction factor.
5. Merge the correction factor into the FP8 micro-scale with stochastic rounding.
6. Store the result as NVFP4 values plus micro-scales plus global scale.

The estimator has much lower quantization error than direct per-element stochastic rounding while retaining approximate unbiasedness.

Cancellation of Rotations in Matrix Multiplication

A major practical observation is that inverse rotations do not need to be explicitly applied in many matrix multiplications. Consider a matrix product

$$C = AB.$$

If the same orthogonal rotation R is applied along the contracted dimension, then

$$(AR^\top)(RB) = A(R^\top R)B = AB.$$

Thus, the rotations cancel algebraically. This is essential: it allows the method to use randomized rotations to improve quantization without paying for explicit inverse rotations in every output.

For linear layer forward and backward computations, the important matrix products are

$$Y = XW^\top,$$

$$\nabla_X = \nabla_Y W,$$

$$\nabla_W = \nabla_Y^\top X.$$

quartet v2 applies rotations along inner dimensions so that they cancel inside these GEMM operations.

Randomized Hadamard Rotations

Fully random orthogonal matrices are too expensive to sample and apply. quartet v2 uses structured randomized Hadamard transforms. A Hadamard matrix of size 2 is

$$H_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

Larger Hadamard matrices are built recursively using the Kronecker product:

$$H_{2n} = \begin{bmatrix} H_n & H_n \\ H_n & -H_n \end{bmatrix}.$$

A normalized Hadamard matrix is orthogonal:

$$\overline{H}_n = \frac{1}{\sqrt{n}} H_n, \qquad \overline{H}_n^\top \overline{H}_n = I.$$

Randomization is introduced by multiplying columns or rows by random signs:

$$R_\omega = \overline{H}_n D_\omega,$$

where D_ω is diagonal with entries in $\{-1, +1\}$. This gives a structured but randomized orthogonal transform.

The lecture emphasized that the random Hadamard distribution is not a truly uniform Haar rotation. However, for group size 128, it works well enough in

practice: the gradient estimator empirically concentrates toward the unquantized gradient as the number of random samples increases.

Visual Concept

Draw a 128-element vector entering a block labeled "random signs", followed by a structured Hadamard butterfly. Then split the output into eight groups of 16 values, each receiving one NVFP4 micro-scale.

Hardware-Level Explanation

Where FP4 Helps

Large language model training is dominated by linear layers. Each linear layer contains GEMM-like operations in the forward and backward passes. For a matrix multiplication

$$C_{M\times N} = A_{M\times K} B_{K\times N},$$

the arithmetic cost is

$$\text{FLOPs} \approx 2MNK.$$

Using FP4 tensor cores can substantially increase the peak throughput of this operation. However, to use FP4 GEMM, tensors must first be converted into the hardware layout expected by tensor cores, including FP4 values and scale metadata.

Why Quantization Kernels Matter

The training step is not only GEMM. For quartet v2, each low precision linear layer may need kernels that perform:

- dequantization from FP4 to BF16 or FP32;
- randomized Hadamard transform;
- transpose or layout conversion;
- absolute maximum reduction;
- FP4 quantization;
- Eden correction computation;
- stochastic rounding of FP8 scales;
- writing NVFP4 values and scales in tensor-core-friendly layout.

If implemented as separate kernels, each stage writes to and reads from global memory. That destroys the advantage of FP4 because global memory traffic dominates.

The engineering goal is therefore kernel fusion:

$$\text{load once} \rightarrow \text{transform} \rightarrow \text{quantize} \rightarrow \text{write compact output.}$$

SM-Level View

Inside an NVIDIA GPU, work is organized into thread blocks. A thread block is scheduled onto an SM. Threads execute in warps of 32 threads. Tensor core operations are issued by warps or warp groups. Shared memory and registers are used to hold tiles between global memory and tensor core operations.

For quartet v2's quantization kernels, the important resources are:

- **Registers:** hold per-thread fragments, scales, random numbers, and intermediate values.
- **Shared memory:** stages tiles for tensor core operations and layout conversion.
- **Tensor cores:** apply small dense matrix products related to the Hadamard transform.
- **CUDA cores and ALUs:** perform format conversions, scale arithmetic, comparisons, bit manipulation, and correction computations.
- **Global memory:** stores FP4 values, FP8 scales, global scales, and intermediate BF16 scale tensors.

A surprising result from the lecture is that the key quantization kernel is neither clearly memory-bound nor tensor-core-bound. Inputs and outputs are tiny because they are FP4, but the kernel performs many conversions and scalar operations. Thus, ALU and instruction overhead are major bottlenecks.

Hadamard Transform Implementation

A naive implementation treats the 128×128 Hadamard matrix as a dense matrix and multiplies it by input tiles using tensor cores. This is easy but wasteful because the matrix is highly structured.

The Hadamard matrix can be decomposed into smaller repeated sign patterns. In the lecture, the implementation exploits structure so that instead of performing all 8×8 tile products, it performs only the necessary small products and then combines them with additions and subtractions.

Conceptually, if the input vector is split into chunks

$$x = \begin{bmatrix} x_1 & x_2 & \cdots & x_8 \end{bmatrix}^\top,$$

then the transform has repeated submatrices $A, B, C, \ldots$ with sign changes. Instead of computing every signed block multiply independently, compute base products

$$Ax_1, \quad Bx_2, \quad Cx_3, \quad \ldots$$

and then combine them using butterfly additions and subtractions.

Why Group Size 128

The lecture motivates 128 as a practical sweet spot.

- Smaller groups such as 16 or 32 showed worse quality in experiments.
- Larger groups increase kernel cost and implementation complexity.
- 128 aligns well with power-of-two Hadamard structure.
- 128 is large enough to mix outliers and approximate rotation-based unbiasedness.
- 128 can be decomposed into manageable tensor core tiles.

The NVFP4 micro-scale group size remains 16. Thus, each 128-element rotation group contains eight NVFP4 micro-scale groups.

Thread Layout Problem

Tensor core fragments do not naturally arrange the 16 values of an NVFP4 quantization group contiguously inside one thread. A naive mapping may distribute the 16 values across four threads, forcing warp shuffle reductions to compute:

$$m_g = \max_{i \in g} |x_i|,$$

and the Eden inner products:

$$\langle z_g, z_g \rangle, \qquad \langle q_g, z_g \rangle.$$

Warp shuffles are expensive and complicate the kernel.

The implementation uses a clever reinterpretation: instead of forcing the mathematical quantization group to match the physical tensor core layout, it permutes the effective order of values. This is equivalent to multiplying the rotation by a permutation matrix P:

$$R' = PR.$$

Since P is orthogonal and the rotation is random anyway, this does not destroy the important rotation properties. But it makes each thread own a contiguous group of 16 values. This removes many warp shuffles and produces larger, more efficient writes.

Visual Concept

Draw two layouts. In the first, one quantization group is striped across four threads, requiring warp shuffles. In the second, after a virtual permutation, each thread owns one full 16-value group and computes its own scale locally.

Two-Kernel Global Scale Strategy

NVFP4 uses a global scale, but the global absolute maximum after the Hadamard transform is not known until the transform has been computed. Two naive solutions are bad:

1. Write the full BF16 transformed tensor to memory, reduce the maximum, then read it again to quantize.
2. Recompute the Hadamard transform twice: once for the maximum and once for quantization.

quartet v2 uses a compromise. The first kernel computes quantized FP4 values and writes micro-scales in BF16, along with partial maxima. The second kernel reads only the much smaller micro-scale tensor, computes the global scale, and stochastically rounds BF16 scales to FP8.

This reduces repeated memory traffic because the second pass reads scales rather than the full tensor.

Programmatic Dependent Launch

The post-processing kernel can be launched using programmatic dependent launch, so the second kernel begins as soon as its dependencies are ready. This reduces the kernel launch gap that would otherwise appear between two separate kernels.

The main idea is:

Kernel 1 produces BF16 scales $\rightarrow$ Kernel 2 converts BF16 scales to FP8 scales.

Without dependency-aware launch, there may be a few microseconds of idle gap. At small sizes, this gap is significant.

Mathematical Reasoning

Unbiased Quantization Definition

Let $x \in \mathbb{R}^d$, and let ω be the randomness used by a quantizer. A quantizer Q is unbiased if

$$\mathbb{E}_\omega \left[Q(x, \omega) \right] = x.$$

For a model with loss L, the gradient estimator after quantized backward propagation should ideally satisfy

$$\mathbb{E}_\omega \left[\widehat{\nabla_\theta L} \right] = \nabla_\theta L.$$

Bias can accumulate across layers because backpropagation composes many quantized operations.

NVFP4 Scaling Constants

The lecture mentioned important constants:

- 6: approximate maximum representable FP4 magnitude.
- 448: maximum representable E4M3 FP8 magnitude.
- 16/17: safety factor used to avoid clipping after FP8 scale rounding.

The purpose of these constants is to make sure that after scaling and rounding, the FP4 value does not exceed its representable range. Clipping destroys unbiasedness because once values saturate, no choice of stochastic rounding probabilities can recover the original expectation.

A simplified non-clipping requirement is

$$\left| \frac{x_i}{S s_g} \right| \leq 6.$$

If s_g is rounded downward too aggressively, then this inequality can fail. The safety factor reduces this risk.

Error of Stochastic Rounding

For $x \in [a, b]$, stochastic rounding has variance

$$\mathrm{Var}[Q(x)] = (x - a)(b - x).$$

This is largest near the midpoint. The expected squared error is

$$\mathbb{E}\left[(Q(x) - x)^2\right] = (x - a)(b - x).$$

In FP4, $b - a$ can be large, so this error can be substantial. Round-to-nearest has squared error

$$(Q_{\mathrm{RTN}}(x) - x)^2 \leq \frac{(b - a)^2}{4},$$

but it is biased. quartet v2 aims to combine low round-to-nearest error with approximate unbiasedness.

Rotation-Based Colinearity

Let R_ω be drawn from a symmetric distribution over rotations. Define

$$\widehat{x}_\omega = R_\omega^\top Q(R_\omega x).$$

For each rotation producing an error component perpendicular to x, symmetry implies the existence of an equally likely opposite rotation producing the mirrored error. Therefore perpendicular components cancel in expectation:

$$\mathbb{E}_\omega\left[\widehat{x}_\omega - \mathrm{proj}_x(\widehat{x}_\omega)\right] = 0.$$

Thus,

$$\mathbb{E}_\omega\left[\widehat{x}_\omega\right] = \alpha x.$$

The remaining issue is correcting α.

Eden Correction Factor

For $z = R_\omega x$ and $q = Q(z)$, define

$$c = \frac{\langle z, z\rangle}{\langle q, z\rangle}.$$

Then

$$\langle cq, z\rangle = c\langle q, z\rangle = \langle z, z\rangle.$$

This makes the corrected vector have the correct projection along z. After rotating back, the estimator is

$$\widehat{x} = R_{\omega}^{\top} cq.$$

In practice, quartet v2 computes such corrections at the micro-scale group level rather than the whole tensor level.

Groupwise Correction

Let g be a group of 16 values inside a 128-value rotation group. Let z_g be the rotated full precision vector for that group, and let q_g be the dequantized vector after FP4 round-to-nearest but before correction. A groupwise correction is

$$c_g = \frac{\langle z_g, z_g \rangle}{\langle q_g, z_g \rangle}.$$

The corrected group is

$$\widetilde{q}_g = c_g q_g.$$

In NVFP4, this is implemented by folding c_g into the FP8 scale:

$$\widetilde{s}_g \approx \mathrm{SR}_{\mathrm{FP8}}(c_g s_g),$$

where $\mathrm{SR}_{\mathrm{FP8}}$ denotes stochastic rounding to FP8.

Rotation Cancellation in GEMM

For two tensors A and B multiplied along an inner dimension, apply the same rotation R to that dimension:

$$\widetilde{A} = AR^{\top}, \qquad \widetilde{B} = RB.$$

Then

$$\widetilde{A}\widetilde{B} = AR^{\top}RB = AB.$$

This identity is the reason randomized rotations can be used without explicitly de-rotating every tensor before the GEMM output.

Arithmetic Intensity

Arithmetic intensity is

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes Moved}}.$$

For a GEMM $C = AB$,

$$\text{FLOPs} \approx 2MNK.$$

If A, B, and C are stored in b-byte elements, the naive memory traffic is roughly

$$\text{Bytes} \approx b(MK + KN + MN).$$

For FP4, the data bytes are small, but scale metadata and quantization kernels add overhead. The real arithmetic intensity of the full training step depends heavily on how much dequantization and requantization are fused.

Speedup Bound

Let T_{BF16} be the time for BF16 training and T_{FP4} be the time for FP4 training. Decompose FP4 time as

$$T_{\text{FP4}} = T_{\text{FP4 GEMM}} + T_{\text{quant}} + T_{\text{other}}.$$

Even if

$$T_{\text{FP4 GEMM}} \approx \frac{1}{4} T_{\text{BF16 GEMM}},$$

the total speedup is limited by the non-GEMM overheads:

$$\text{Speedup} = \frac{T_{\text{BF16}}}{T_{\text{FP4 GEMM}} + T_{\text{quant}} + T_{\text{other}}}.$$

This explains why the lecture reports more modest end-to-end speedups than the peak tensor core ratio.

Code Walkthrough

Minimal Stochastic Rounding

```
def stochastic_round_scalar(x, grid):
    # Find lower and upper representable values.
    a = max(v for v in grid if v <= x)
    b = min(v for v in grid if v >= x)

    if a == b:
        return a

    # Probability of rounding upward.
    p_up = (x - a) / (b - a)

    # Randomly choose upper or lower grid point.
    u = random_uniform_0_1()
    if u < p_up:
        return b
    else:
        return a
```

This code captures the scalar idea behind stochastic rounding. The probability is chosen so that the expectation equals x. On a GPU, the random number would usually come from a counter-based generator such as Philox. The actual implementation cannot call Python functions; it must generate random bits per thread, preferably reusing each random generator call across multiple scale values.

Simplified NVFP4 Quantization

```
def nvfp4_quantize_group(x_group, global_scale):
    # x_group has 16 values.
    # local_scale is chosen so the largest value maps near FP4 max.
    fp4_max = 6.0
    local_absmax = max(abs(v) for v in x_group)

    if local_absmax == 0.0:
        scale = 0.0
    else:
        scale = local_absmax / (global_scale * fp4_max)

    # In real NVFP4 this scale is rounded to E4M3 FP8.
    scale_fp8 = round_to_fp8(scale)

    q = []
    for v in x_group:
```

```
        y = v / (global_scale * scale_fp8)
        q.append(round_to_fp4_nearest(y))

    return q, scale_fp8
```

The important hardware behavior is that the 16-value group shares a scale. If adjacent threads do not own contiguous groups, the local absolute maximum reduction requires warp-level communication. quartet v2 avoids this by choosing a virtual layout where each thread owns a full micro-scale group.

Simplified Eden-Style Group Correction

```
def eden_correct_scale(z_group, q_dequant_group, scale):
    # z_group is the rotated high precision vector.
    # q_dequant_group is the dequantized FP4 approximation.
    numerator = 0.0
    denominator = 0.0

    for z, q in zip(z_group, q_dequant_group):
        numerator += z * z
        denominator += q * z

    if denominator == 0.0:
        correction = 1.0
    else:
        correction = numerator / denominator

    corrected_scale = correction * scale
    return corrected_scale
```

The numerator and denominator are inner products. In the real kernel, these operations are performed by per-thread registers and fused multiply-add instructions. The correction is folded into the micro-scale, not the FP4 values.

Randomized Hadamard Transform

```
def randomized_hadamard_128(x, signs):
    # x has length 128.
    # signs contains +1 or -1 values.
    y = [x[i] * signs[i] for i in range(128)]

    # In-place Walsh-Hadamard butterfly.
    h = 1
    while h < 128:
        for i in range(0, 128, 2 * h):
            for j in range(i, i + h):
                a = y[j]
```

```
                b = y[j + h]
                y[j] = a + b
                y[j + h] = a - b
        h *= 2

    # Normalize to make the transform orthogonal.
    inv_sqrt_128 = 1.0 / math.sqrt(128.0)
    return [v * inv_sqrt_128 for v in y]
```

This is the conceptual transform. The actual kernel does not necessarily implement this exact butterfly form. The lecture explained that the implementation may use tensor core operations on structured blocks and then combine partial products with additions and subtractions.

Fused Quantization Kernel Skeleton

```
__global__ void fused_rotate_quantize_kernel(
    const uint8_t* fp4_in,
    const uint8_t* fp8_scales_in,
    uint8_t* fp4_out,
    __nv_bfloat16* bf16_scales_out,
    float* partial_absmax,
    uint64_t seed,
    int rows,
    int cols
) {
    // 1. Load FP4 values and FP8 scales.
    // 2. Dequantize into BF16 or FP32 fragments.
    // 3. Apply randomized Hadamard transform.
    // 4. Compute local absmax for groups of 16.
    // 5. Round transformed values to FP4.
    // 6. Dequantize again to compute Eden correction.
    // 7. Store FP4 values and BF16 corrected scales.
    // 8. Write partial maxima for global scale reduction.
}
```

The kernel is designed to minimize global memory traffic. The expensive intermediate values after dequantization and rotation should live in registers or shared memory, not global memory.

Approximate Reciprocal Footgun

The lecture emphasized a real bug caused by approximate reciprocal with flush-to-zero behavior. A simplified dangerous pattern is:

```
float safe_inverse(float x) {
    if (x > 0.0f) {
```

```
        return approximate_reciprocal_ftz(x);
    } else {
        return 0.0f;
    }
}
```

This looks safe, but if x is positive and subnormal, the approximate reciprocal instruction may flush it to zero internally. Then the hardware effectively computes

$$\frac{1}{0},$$

which can produce infinity and later NaNs. The fix is to guard against very small values using a threshold above the flush-to-zero boundary:

```
float safer_inverse(float x) {
    const float eps = 1.0e-30f;
    if (x > eps) {
        return approximate_reciprocal_ftz(x);
    } else {
        return 0.0f;
    }
}
```

The broader lesson is that fast math instructions are not algebraically identical to precise floating point operations. For low precision training, small numerical corner cases can produce training divergence.

Post-Processing Kernel for FP8 Scale Rounding

```
__global__ void round_bf16_scales_to_fp8_kernel(
    const __nv_bfloat16* bf16_scales,
    uint8_t* fp8_scales,
    float* global_scale,
    uint64_t seed,
    int num_scales
) {
    // Each thread processes multiple scales.
    // Generate random bits using a counter-based RNG.
    // Use stochastic rounding from BF16-like scale to E4M3 FP8.
    // Pack stores so each thread writes more than a single byte.
}
```

Thread coarsening matters because a single random number generation call produces many bits. If one thread used a full random generator call for one FP8 scale, random generation overhead would dominate. Processing multiple scales per thread amortizes this cost.

Performance Intuition

Why Per-Element FP4 Stochastic Rounding Is Costly

Per-element FP4 stochastic rounding is unbiased but high variance. For a coarse grid, the distance between adjacent representable values is large. Therefore, each stochastic decision injects substantial noise. Since every element is rounded independently, the tensor-level noise can be large.

quartet v2 reduces error by using round-to-nearest for the FP4 values and placing stochasticity at the FP8 scale level. The stochastic operation is:

$$s_g \leftarrow \mathrm{SR}_{\mathrm{FP8}}(c_g s_g),$$

instead of

$$q_i \leftarrow \mathrm{SR}_{\mathrm{FP4}}\left(\frac{x_i}{S s_g}\right) \quad \text{for every } i.$$

This is much less noisy.

Why Forward and Backward Passes Differ

The forward pass affects activations and model outputs directly. Bias is less clearly fatal than in the backward pass, but quantization error can still hurt loss. quartet v2 uses the 4/6 heuristic on the forward pass, which tries two scaling choices and keeps the one with lower error.

The backward pass is more sensitive to bias because it controls optimizer updates. quartet v2 therefore emphasizes unbiased gradient estimation for backward quantization.

The Four-over-Six Heuristic

The 4/6 heuristic compares two scale choices for a group. Since FP4 has prominent values around 4 and 6, choosing whether the maximum maps to 4 or 6 can affect error. A simplified version is:

$$s_g^{(6)} = \frac{m_g}{6}, \qquad s_g^{(4)} = \frac{m_g}{4}.$$

The method computes both candidate quantizations and chooses the one with lower error:

$$s_g = \arg \min_{s \in \{s_g^{(4)}, s_g^{(6)}\}} \sum_{i \in g} \left(x_i - s Q_{\mathrm{FP4}}\left(\frac{x_i}{s}\right)\right)^2.$$

This does not guarantee unbiasedness, so quartet v2 uses it mainly in the forward pass.

Why Requantizing Weights Can Help

NVIDIA's earlier NVFP4 recipe used square block weight quantization so that the same quantized weight layout could be reused for both forward and transposed backward matrix multiplications. This avoids requantizing weights.

quartet v2 requantizes weights in the backward pass because the backward quantization needs rotations along the appropriate inner dimension. Although this adds quantization work, it enables native NVFP4 scaling instead of square block scaling and allows the 4/6 heuristic to be applied more effectively. The net quality improvement is positive.

Kernel Bottlenecks

The lecture's Nsight Compute discussion suggested the fused quantization kernel has unusual bottlenecks:

- Tensor core utilization is not the main limiter.
- Memory bandwidth utilization is low because FP4 inputs and outputs are tiny.
- ALU and conversion instructions dominate.
- Work is bursty: different phases stress different units.
- Register pressure limits occupancy.
- Short scoreboard and math throttle stalls appear because dependencies and unit contention are significant.

This is unlike standard GEMM, which is usually tensor-core-bound or memory-bandwidth-bound depending on shape and reuse.

Why End-to-End Speedup Is Modest

The lecture mentioned end-to-end speedups around 25% to 35% over BF16 for a reported multi-billion-parameter setup, not a full $4\times$. Reasons include:

- quantization and dequantization kernels are expensive;
- PyTorch and compilation overhead remain;
- custom kernels may inhibit some compiler optimizations;
- small kernels suffer from launch overhead;
- not all model operations are FP4 GEMMs;
- optimizer states and some computations remain higher precision.

The larger the model, the more GEMM dominates, so larger models may see better relative speedups.

Numerical Stability and NaNs

A key engineering lesson is that low precision systems can fail due to details that seem unrelated to the high-level algorithm. The approximate reciprocal flush-to-zero bug caused NaNs in long training. This shows that FP4 training requires not only mathematical unbiasedness, but also careful handling of:

- subnormal values;
- division by tiny numbers;
- overflow and clipping;
- random number generator quality;
- scale underflow;
- format conversion semantics.

Common Misconceptions

Misconception: FP4 Training Is Just Casting Tensors to Four Bits

FP4 training is not a simple cast. The recipe requires groupwise scales, global scales, randomized rotations, correction factors, and custom kernels. Without these, training quality degrades or becomes unstable.

Misconception: Unbiasedness Alone Solves the Problem

Unbiasedness is necessary for stable gradient estimation, but an unbiased estimator can still have very high variance. Per-element FP4 stochastic rounding is unbiased but noisy. quartet v2 improves the variance-error tradeoff.

Misconception: Hadamard Rotations Are Free

Hadamard rotations are cheaper than dense random rotations, but they are not free. They require tensor core operations, additions, subtractions, data layout management, and extra kernel logic. The implementation must exploit structure to avoid making the rotation cost dominate.

Misconception: The Global Scale Can Be Computed Without Cost

The global scale depends on the transformed tensor. Computing it naively requires either writing the transformed tensor to memory or recomputing the

transform. quartet v2 uses a two-kernel strategy to reduce this overhead.

Misconception: Fast Math Instructions Are Always Safe

Fast approximate reciprocal can flush subnormal values to zero and produce infinities or NaNs. Low precision training amplifies these corner cases because scales can become tiny.

Misconception: Tensor Core Utilization Is the Only Metric That Matters

For the fused quantization kernel, tensor core utilization is relatively low. The bottleneck is often instruction overhead, data type conversion, scale arithmetic, and register pressure. Optimizing FP4 training requires looking beyond GEMM throughput.

Misconception: The Same Quantized Weight Should Always Be Reused

Reusing weights avoids requantization, but it may force worse scaling geometry. quartet v2 shows that requantizing weights in the backward pass can improve quality enough to justify the additional work.

Practical Takeaways

1. **Backward-pass quantization must be treated carefully.** Bias in gradients is more dangerous than ordinary forward-pass approximation error.
2. **Stochastic rounding should be placed where it is least harmful.** quartet v2 moves stochastic rounding from FP4 elements to FP8 micro-scales.
3. **Randomized rotations are both an outlier-smoothing tool and an unbiasedness tool.** They reduce the impact of outliers and make quantization error symmetric.
4. **Group size matters.** 128-element rotation groups are a practical compromise between quality and kernel cost.
5. **NVFP4 metadata is part of the algorithm.** The FP8 scales and global scale are not incidental; they are where correction factors and range control live.
6. **Kernel fusion is mandatory.** Separate dequantize, rotate, transpose, reduce, and quantize kernels would move too much data through global memory.

7. **Thread layout can be an algorithmic choice.** Reinterpreting the tensor core fragment layout as a permutation of the rotation can remove warp shuffles.
8. **Low precision training is a systems problem.** Mathematical estimators, number formats, kernel scheduling, memory layout, and framework overhead all interact.

Project or Experiment Ideas

Experiment 1: Scalar Stochastic Rounding Error

Implement stochastic rounding and round-to-nearest for a toy FP4 grid. Sample values from a Gaussian distribution and measure:

$$\mathbb{E}[Q(x) - x], \qquad \mathbb{E}\left[(Q(x) - x)^2\right].$$

Verify that stochastic rounding has near-zero bias but higher mean squared error.

Experiment 2: Random Rotation Quantization

Generate random vectors $x \in \mathbb{R}^{128}$. Compare:

- direct round-to-nearest quantization;
- direct stochastic rounding;
- randomized Hadamard plus round-to-nearest;
- randomized Hadamard plus Eden correction.

Measure angular error:

$$\cos\theta = \frac{\langle x, \widehat{x} \rangle}{\|x\|_2 \|\widehat{x}\|_2}.$$

Also measure bias by averaging over many random sign patterns.

Experiment 3: Group Size Ablation

Repeat the rotation experiment for group sizes

$$16, \quad 32, \quad 64, \quad 128, \quad 256.$$

Measure how quickly the estimator concentrates toward the original vector as the number of random samples increases.

Experiment 4: Tiny CUDA Quantization Kernel

Write a CUDA kernel that quantizes groups of 16 FP32 values to a toy FP4 grid with one FP32 scale per group. Compare two layouts:

1. group values distributed across multiple threads;
2. one thread owns the full group.

Profile the difference in warp shuffles, memory stores, and instruction count.

Experiment 5: Hadamard Transform Implementations

Implement a 128-point randomized Hadamard transform in three ways:

- naive dense matrix multiplication;
- butterfly additions and subtractions;
- tensor-core block products plus structured recombination.

Compare runtime, register usage, and numerical accuracy.

Experiment 6: Approximate Reciprocal Failure

Write a CUDA microbenchmark that computes approximate reciprocal on very small positive values. Check when values flush to zero and produce infinity. Use this to understand why scale arithmetic needs explicit lower bounds.

Experiment 7: Forward Quantization with Four-over-Six

Implement the 4/6 heuristic on random tensors and transformer-like activation distributions. Compare quantization error against ordinary absmax scaling.

Final Mental Model

quartet v2 should be understood as a complete low precision training recipe, not a single quantizer. The core idea is:

> Use randomized Hadamard rotations to make FP4 quantization errors symmetric, use Eden-style correction to recover unbiased gradient estimates, place stochastic rounding only on FP8 micro-scales to reduce error, and fuse the entire dequantize-rotate-quantize pipeline into GPU kernels that avoid unnecessary global memory traffic.

The mathematical story is about preserving the expected gradient direction under extreme quantization. The hardware story is about making that estimator cheap enough to run inside real LLM training. The systems story is that FP4 tensor cores alone do not deliver speedups unless quantization, scaling, layout, randomness, and kernel launch overhead are engineered as carefully as the GEMMs themselves.